

Project Design Phase — Solution Architecture

Script-Controlled ACL — Restrict Record Access Based on Field Value

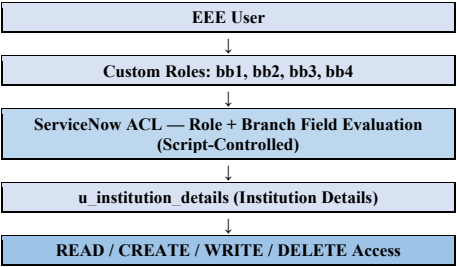
Solution Architecture

Bridges the business need to restrict unauthorized record access with a technical ServiceNow implementation using ACLs, user roles, record field values, and script-controlled access logic to enforce record-level security.

Only users holding the required role can view records where Branch = EEE; administrators retain full access. Data lives in the custom table u_institution_details (Student Roll Number, Student Name, Faculty Name, Branch, Email, Phone Number, Description; Branch = ECE/EEE/CSE).

Custom roles bb1 (Read), bb2 (Create), bb3 (Write), bb4 (Delete) are assigned to the EEE User. The READ ACL combines the bb1 role, the Branch is EEE data condition, and an advanced script; CREATE/WRITE/DELETE ACLs use bb2/bb3/bb4. Verified by impersonating users.

Architecture Flow



Architecture Components

- User & Role Layer: EEE User created and assigned custom roles bb1–bb4.
- Data Layer: u_institution_details stores institution/student records with Branch values ECE, EEE, CSE.
- Security Layer: ServiceNow ACLs control READ, CREATE, WRITE, DELETE operations.
- Decision Layer: READ ACL evaluates admin + bb1 roles via advanced script; Branch is EEE restricts applicable records.
- Verification Layer: Validated by impersonation — bb1 sees EEE records, unauthorized users see none, admins see all.

Access Flow

READ Access	CREATE Access	WRITE Access	DELETE Access	Administrator
bb1 + Branch is EEE + Script	bb2	bb3	bb4	—
↓	↓	↓	↓	↓
EEE Branch Records Accessible	Create Permission	Edit Permission	Delete Permission	Full Access to All Records

ACL Evaluation Logic

1. User requests access to a record
2. ServiceNow finds matching table + field ACL rules (table → parent table → *)
3. ACL permissions evaluated (role + script + data condition)
4. Pass → Grant access to the record | Fail → Deny access to the record